RTP (Real-Time Payment) Transaction Validator – Development Document

Java / Spring Boot Use Case

---

1. Introduction

The **RTP Transaction Validator System (RTVS)** is a backend service that simulates a **real-time account-to-account payment system**, similar to UPI-style instant payments or U.S. RTP / FedNow rails.

The system validates payment requests in real time, enforces balance and limit controls, posts ledger entries atomically, and provides full transaction traceability.

This use case exposes fresh talent to **core banking payment flows**, including validation, authorization, ledger posting, and auditability.

This document outlines:

- Business Context
- System Architecture
- Functional Requirements
- Business Rules & Validation Logic
- API Endpoints
- Security Configuration
- Error Handling
- Database Design
- Non-Functional Requirements

---

2. Business Context

In real-time payments:

- Money moves **instantly**
- Transactions are **irrevocable**

- Errors must be caught **before posting**
- Ledger integrity is critical

Banks operating instant payment systems must:

- Validate sender balance
- Validate receiver existence
- Enforce daily transaction limits
- Prevent duplicate or replayed requests
- Maintain accurate real-time ledger balances

This system represents the **bank's payment initiation and validation layer** for RTP payments.

---

3. System Architecture

3.1 Overview

RTVS will be built as a **Spring Boot RESTful microservice** responsible for:

- Accepting RTP payment requests
- Validating business rules
- Posting ledger entries atomically
- Persisting transaction history
- Providing balance and audit APIs

3.2 High-Level Components

- **Payment API Controller**
- **Validation Engine**
- **Ledger Posting Service**
- **Transaction Repository**
- **Balance Service**
- **Audit & Failure Logger**

---

4. Security and Authentication

4.1 Authentication

- JWT-based authentication

- All endpoints require valid JWT token
- Tokens include `userId` and `role`

4.2 Role-Based Authorization (RBAC)

Roles:

- **USER** – Initiate payments, view own balance and history
- **ANALYST** – View transaction and failure logs
- **ADMIN** – Configure limits and system parameters

Access Rules:

- Users can only view their own data
- Analysts have read-only access to all transactions
- Admins manage limits and configurations

---

5. Domain Concepts

5.1 User Account

Represents a bank customer with a real-time balance.

Attributes:

- user_id
- current_balance
- daily_limit
- status (ACTIVE / BLOCKED)

5.2 RTP Transaction

Represents a real-time payment request.

Attributes:

- transaction_id
- payment_request_id (idempotency key)
- sender_id
- receiver_id

- amount
- status
- failure_reason
- created_at

5.3 Ledger Entry

Represents a financial debit or credit.

Attributes:

- ledger_id
- user_id
- transaction_id
- entry_type (DEBIT / CREDIT)
- amount
- balance_after
- created_at

---

6. Functional Requirements

6.1 Payment Initiation

- System must accept RTP payment requests via API
- Requests must be idempotent using `paymentRequestId`
- Duplicate requests must not cause double debit

6.2 Validation Rules

The system must validate:

1. Sender exists and is ACTIVE
2. Receiver exists and is ACTIVE
3. Amount > 0
4. Sender has sufficient balance
5. Sender daily limit not exceeded

6.3 Ledger Posting

- Debit sender account
- Credit receiver account
- Ledger posting must be **atomic**
- Balance updates must be consistent

6.4 Transaction Tracking

- Store all transactions (success & failure)
- Capture failure reason codes
- Maintain full audit trail

---

7. Business Logic

7.1 Validation Flow

1. Receive payment request
2. Check idempotency (paymentRequestId)
3. Validate sender account
4. Validate receiver account
5. Validate amount
6. Validate sender balance
7. Validate daily limit
8. Post ledger entries
9. Mark transaction COMPLETED

If any step fails → mark transaction REJECTED.

7.2 Daily Limit Calculation

```
daily_total = sum(amount)
              where sender_id = X
              and transaction_date = today
              and status = COMPLETED
```

If:

```
daily_total + new_amount > daily_limit
```

→ reject transaction.

7.3 Transaction States

State	Meaning
INITIATED	Request received
VALIDATED	Passed validations
COMPLETED	Ledger posted successfully
REJECTED	Failed validation or posting

7.4 Failure Reason Codes

- INSUFFICIENT_FUNDS
- RECEIVER_NOT_FOUND
- SENDER_NOT_FOUND
- DAILY_LIMIT_EXCEEDED
- DUPLICATE_REQUEST
- INVALID_AMOUNT
- ACCOUNT_BLOCKED

---

8. API Endpoints

8.1 RTP Payment API

Initiate Payment

POST `/api/v1/rtp/payments`

Request:

```
{
  "paymentRequestId": "PR-10001",
  "senderId": "U100",
  "receiverId": "U200",
  "amount": 150.00,
  "currency": "USD",
  "note": "Dinner split"
}
```

Success Response:

```
{
```

```
  "transactionId": "TX-90001",
  "status": "COMPLETED",
  "postedAt": "2026-02-03T20:10:00Z"
}
```

Failure Response:

```
{
  "transactionId": "TX-90002",
  "status": "REJECTED",
  "reasonCode": "INSUFFICIENT_FUNDS",
  "message": "Sender does not have sufficient balance"
}
```

8.2 Balance API

GET `/api/v1/rtp/users/{userId}/balance`

Response:

```
{
  "userId": "U100",
  "currentBalance": 1250.75,
  "dailyLimit": 2000.00
}
```

8.3 Transaction History API

GET `/api/v1/rtp/users/{userId}/transactions?from=YYYY-MM-DD&to=YYYY-MM-DD`

Response:

```
{
  "transactions": [
    {
      "transactionId": "TX-90001",
      "amount": 150.00,
      "status": "COMPLETED",
      "direction": "DEBIT",
      "timestamp": "2026-02-03T20:10:00Z"
    }
  ]
}
```

8.4 Failed Transactions API (Analyst/Admin)

GET `/api/v1/rtp/transactions/failed?from=...&to=...`

Response:

```
{
  "failedTransactions": [
    {
      "transactionId": "TX-90002",
      "senderId": "U100",
      "amount": 500.00,
      "reasonCode": "DAILY_LIMIT_EXCEEDED"
    }
  ]
}
```

---

9. Database Design Skipped

---

10. Error Handling

Standard Error Response

```
{
  "timestamp": "2026-02-03T20:10:00Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Daily limit exceeded",
  "path": "/api/v1/rtp/payments"
}
```

Common Errors

HTTP Code	Cause
400	Validation failure
401	Unauthorized
403	Forbidden
404	User not found
409	Duplicate request
500	Internal error

---

11. Non-Functional Requirements

- Atomic transactions (ACID)

- Idempotent payment initiation
- Handle concurrent payment requests safely
- Low latency (< 200ms)
- Full audit trail
- Minimum 80% unit test coverage

---

12. Deliverables

- Spring Boot REST service
- Ledger-based payment logic
- API documentation (Swagger)
- Unit and integration tests
- Sample test data
- Architecture and flow diagrams